

第一章基础概念:

图形系统的基本构成:

输入设备 → CPU → GPU → **帧缓存** → 输出设备, 图像是在帧缓存中形成的

OpenGL 渲染管线: 顶点数据 → 顶点着色器 → 细分控制着色器 → 细分计算着色器 → 几何着色器 → 图元设置 → 裁切 → 光栅化 → 片元着色器

重点——完整讲述每一个步在做什么? 这三个结合起来记忆

1. **顶点数据**: 顶点属性数组 (**位置、法向、颜色、纹理坐标**), CPU 把数据放到 GPU 缓冲 (VBO/VAO)

2. **顶点着色器**: 核心任务是坐标变换, **单独处理每一个顶点**, 把顶点从对象/世界/相机空间变到裁剪空间 (屏幕), 输出 `gl_Position`; 同时把需要插值到片元阶段的属性输出(out)出去——让后续阶段知道这个顶点在哪

3. 【了解】 **细分控制着色器** -- **细分计算着色器** (让模型更平滑) --> **几何着色器** (能改变图元的类型)

4. **图元设置**: 根据设定的图元类型 (`GL_TRIANGLES`), 把这些点组装成线段或三角形

5. **裁切[常考概念]**: 切掉视见体 (相机能看到的盒子) 之外的东西, 只保留在屏幕里的部分

6. **光栅化**: 纯数学的几何图形 (三角形) 变成屏幕上离散的片元 (Fragment)

7. **片元着色器**: 核心任务是**上色**, 计算出每个像素最终应该显示什么颜色——光照计算 (Phong 模型)、纹理采样

像素：图像的基本单元

光栅：像素的阵列（屏幕/帧缓存本质是光栅）

图像合成的三/四要素：对象，观察者，光源各自**完全独立** + 材质

对象：包含几何信息（形状和位置）和表面属性（怎么反光）

观察者：就是**虚拟相机**，包含位置+朝向+投影方式+视见体（能看到的范围）

光源：位置或方向，强度与颜色，**光源与材质**共同决定光照射在物体上的效果

虚拟照相机模型：

场景里有对象与观察者（相机），通过“成像系统”得到图像

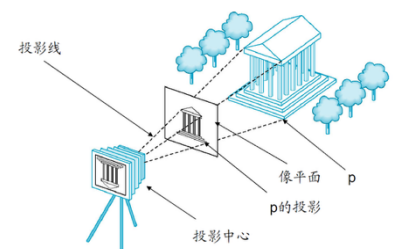
1.物理基础：真实的针孔相机模型，COP 是观察点

2.关键简化：等效的虚拟相机模型，将成像平面从 COP

后方，虚拟地移动到 COP 的前方

3.优点：把对象、观察者和光源区分开

剪裁：相机/视见体只能看到世界的一部分，视见体外对象要裁剪掉（减少计算）



全局和局部光照模型：

全局光照模型：目标是**照片级真实感**，它模拟光线在场景中所有表面之间多次反弹、相互作用的物理过程，但**计算开销极大**，难以用于实时应用。

局部光照模型：目标是**实时/交互式渲染**，它只计算光源直接照射到物体表面的效果，忽略物体间复杂的光线相互作用，因此**计算快**，核心是后面的

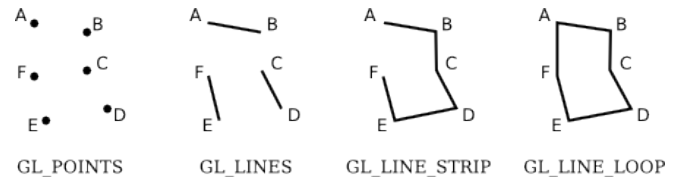
Phong 光照模型

第二章 OpenGL

图元：显卡处理流水线能够理解的几何与图像数据单元，分为几何图元与光栅图元两类；点、线段、三角形、多边形及其组合方式都有对应的数据类型

重点！ 7 种基本图元及其连接方式：

点和线：



1. GL_POINTS: 独立的点
2. GL_LINES: 两个点连成一条线段（点对独立，不连续）
3. GL_LINE_STRIP: 折线（首尾相连，不闭合）
4. GL_LINE_LOOP: 闭合折线（最后一个点自动连回第一个点）

三角形 (Triangles) —— 必考点：

1. GL_TRIANGLES: 每三个顶点定义一个独立的三角形
【顶点数要求：必须是 3 的倍数，绘制：(v0, v1, v2), (v3, v4, v5)
2. GL_TRIANGLE_STRIP (三角形带): 前一个三角形的最后两个顶点与当前新顶点构成新三角形
3. GL_TRIANGLE_FAN: 所有三角形共用第一个顶点 v0 作为中心点

复习：

- 看到 mode = GL_TRIANGLES 能立刻知道：每 3 个顶点组成 1 个三角形
- 看到 TRIANGLE_STRIP / FAN 能知道：后续每加入 1 个顶点，会在已有结构上生成新的三角形（因此同样数量顶点可更省）

多边形的性质：对于一般多边形，必须满足以下条件才能被正确渲染：

1. 简单性：边不相交，且没有空洞
2. 凸性：多边形内任意两点的连线都在多边形内部
3. 平面性：所有顶点在同一平面上

着色器最小实例：

vshader.glsl

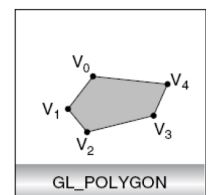
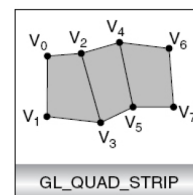
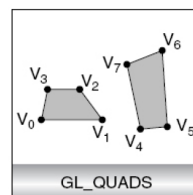
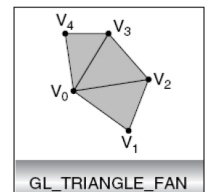
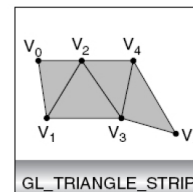
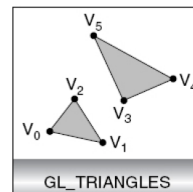
```
#version 330 core
in vec3 vPosition;

void main()
{
    gl_Position = vec4(vPosition, 1.0);
}
```

fshader.glsl

```
#version 330 core
out vec4 fColor;

void main()
{
    fColor = vec4(1.0, 0.0, 0.0, 1.0);
}
```



顶点着色器 vshader:

(1) 顶点属性输入: in vec3 vPosition, 每个顶点会有一份 vPosition

(x,y,z)

(2) 顶点着色器的输出: gl_Position = vec4(vPosition, 1.0), 等价于

(x, y, z, 1), 把三维点扩成齐次坐标, 表示裁剪空间坐标 (x, y, z, w)

片元着色器 fshader:

(1) out vec4 fColor: 片元着色器的颜色输出, 写入到帧缓存的颜色附件

(2) fColor = vec4(1,0,0,1): 对每个片元都输出固定 RGBA, 这里表示纯红色、不透明

VBO: GPU 存储, 本质还是一堆 Bytes, 解决传输开销

VAO: 解释数据 (哪些属性启用, 哪个 VBO, 数据格式[指针,类型,大小,步长,偏移])

早期的 OpenGL 为了将模型的顶点数据传送到显卡，需要逐个顶点进行（冗余处理的问题），如果还需要额外的信息（纹理坐标和法线）的话，当模型比较复杂时，将导致大量函数的调用，**传输开销是相当大的！**为了解决这个问题引入了 VBO(Vertex Buffer Object)，VBO 可以将顶点数据保存在显存中，绘制时直接从显存中取数据，减少了数据传输的开销

通过 VBO 我们可以将顶点属性数据保存在显存中，当绘制时问题又来了，需要调用好几个函数，过程挺复杂的。为了解决这个问题，OpenGL 又引入了 VAO(Vertex Array Object)来关联 VBO 中的数据，有了 VAO，任何数组形式的 GL 函数调用都会添加到 VAO 的绘制列表当中（直到解除 VAO 绑定），当需要绘制的时候，我们仅需要重新绑定 VAO，那么之前创建的绘制列表将会重新激活，使得绘制代码更加简洁。

`glBufferData` 用于向 VBO 传递数据：第二个参数表示数据的大小，第三个参数表示指向数据的指针；`glBufferData(GL_ARRAY_BUFFER, size, data, usage);`

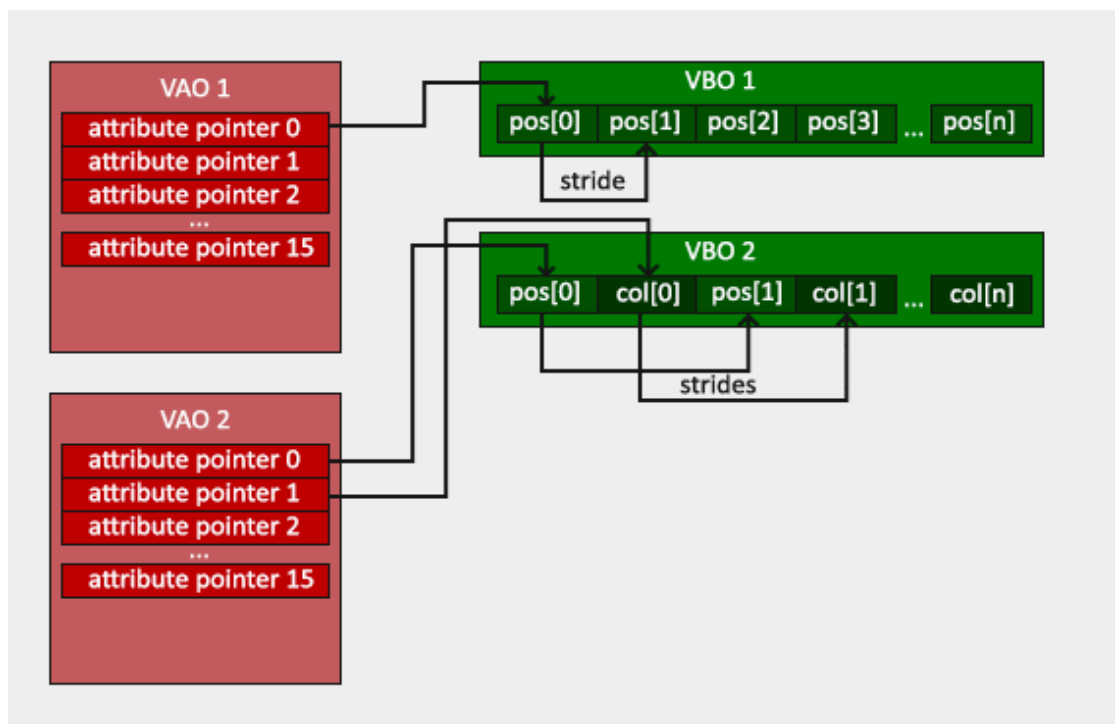
`glVertexAttribPointer` 用于将顶点着色器中的 attribute 变量与 VBO 中的数据进行绑定：第一个参数表示属性的位置，第二个参数表示分量的个数，第三个参数表示数据类型，第四个参数表示是否归一化，第五个参数表示连续顶点属性之间的偏移量，第六个参数表示组件的第一个分量在对应的数组顶点属性中的偏移量

`glVertexAttribPointer(index, size, type, normalized, stride, pointer);`

可以把 VBO 理解成一个数据区域，这里面存放了渲染所需要的一切信息。但需要注意的是，数据在 VBO 里面的时候，OpenGL 是并不知道这里的每个

数据所代表的具体含义的，只是一堆数值罢了。VAO 可以定义一种格式说，告诉顶点着色器只取 VBO 里面的 x,y,z 的值；也可以定义一种格式说，每次只取 VBO 里面的 r,g,b 的值。这样我们就可以在渲染的时候，绑定不同的 VAO，实现按照不同的格式将 VBO 里面的东西渲染出来。

考点：要不要多个 VAO——取决于顶点属性布局不同，代价结构更清楚



第三章：模-视变换

几何对象：三维空间对象，点线面体

几何基本要素：

1. 点：三维空间的一个位置
2. 标量：一个数字，表示大小，充当系数
3. 向量 = 方向 + 长度，没有位置信息

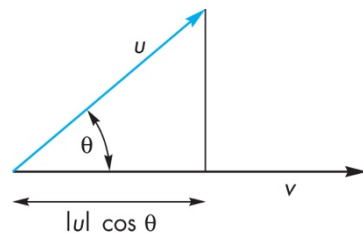
例子： $P(t) = A + t \cdot (B - A)$ A,B 一个点，B-A 是向量，t 是标量

向量空间：又称为线性空间，包含两种实体：向量+标量

经典运算：点积和叉积

点积： $\mathbf{u} \cdot \mathbf{v} = |\mathbf{u}| |\mathbf{v}| \cos \theta$

叉积： $\mathbf{n} = \mathbf{u} \times \mathbf{v}$ ， $\mathbf{n}$ 为向量， $\mathbf{n}$ 的长度等于 $|\mathbf{u}| |\mathbf{v}| \sin \theta$



Step 2: 叉积 (cross product)

$$\vec{N} = \vec{AB} \times \vec{AC}$$

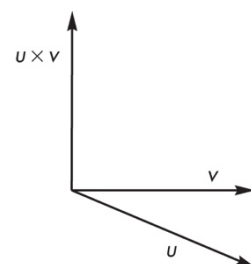
展开为：

$$\vec{N} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ x_2 - x_1 & y_2 - y_1 & z_2 - z_1 \\ x_3 - x_1 & y_3 - y_1 & z_3 - z_1 \end{vmatrix}$$

即：

$$\vec{N} = \begin{pmatrix} (y_2 - y_1)(z_3 - z_1) - (z_2 - z_1)(y_3 - y_1) \\ (z_2 - z_1)(x_3 - x_1) - (x_2 - x_1)(z_3 - z_1) \\ (x_2 - x_1)(y_3 - y_1) - (y_2 - y_1)(x_3 - x_1) \end{pmatrix}$$

这就是该面的法向量（未单位化）。



仿射空间：向量空间的扩展，加入了另一对象：点

运算：

标量-向量乘法 $\rightarrow$ 向量 or 向量-向量加法 $\rightarrow$ 向量

点-向量加法 $\rightarrow$ 点 or 点-点减法 $\rightarrow$ 向量

点 + 点 **没有几何意义**（没法解释成“位置+位置”是什么）

仿射空间核心点是**不随坐标系改变的几何关系**，没有天然原点，对应图形学里：模型放在哪儿、相机放在哪儿，这些都不应该依赖“你把世界原点选在哪里”

直线的参数形式： $X(\alpha) = (1 - \alpha)P + \alpha Q$ ，本质就是点+向量，以某个点为基准，加一个位移，得到点的线性组合——**仿射组合里系数和必须为 1**

更一般的形式： $P = \alpha_0 P_0 + \alpha_1 P_1$ ，其中 $\alpha_0 + \alpha_1 = 1$

仿射加法(一个凸组合)： $A = \alpha_1 P_1 + \alpha_2 P_2 + \dots + \alpha_k P_k$ 且 $\alpha_1 + \alpha_2 + \dots + \alpha_k = 1$

$\alpha \in [0, 1]$ 得到**线段** PQ 上的点（这就是“凸组合”的核心）

凸组合(是一个点): $(1 - \alpha)P + \alpha Q$ 且 $\alpha \in [0, 1]$, 扩展到三角形内部 (含边界)

$$X = \alpha P + \beta Q + \gamma R, \quad \alpha, \beta, \gamma \geq 0, \quad \alpha + \beta + \gamma = 1$$

凸包定理: 如果 $\alpha_i \geq 0$, 那么得到 $\{P_1, P_2, \dots, P_n\}$ 的凸包 (注意这里允许某些点为 0)

凸包定义: 对于一个点集 P , 其凸包是包含 P 中所有点的最小的凸多边形

其实就是**最小凸集**: $\text{conv}(S) = \cap \{ C \mid C \text{ 是凸集且 } S \subseteq C \}$

定理证明: 实际上就是证明定理和定义是等价的——另外一个文档, 思路:

(1) 证明凸组合包含点集 S (2) 证明凸组合是凸集 (3) 证明凸组合是最小的

Section2 几何对象表示

线性无关和维数: 回答空间需要几根“互不重复信息”的轴来描述

- **线性无关**: 没有一根向量能用其他向量“凑出来”
- **维数**: 最少需要几根线性无关向量来张成空间, 3D 维数就是 3

坐标系: 用数字表示向量, 不需要远点, 表示的是位移【向量的概念】

$$w = \mathbf{a}^T \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} = \mathbf{a}^T \mathbf{v}$$

空间由一组**线性无关的基**张成, 一组基向量定义了坐标系

那么 (a_1, a_2, a_3) 就是 $\mathbf{v}$ 在这个坐标系下的坐标

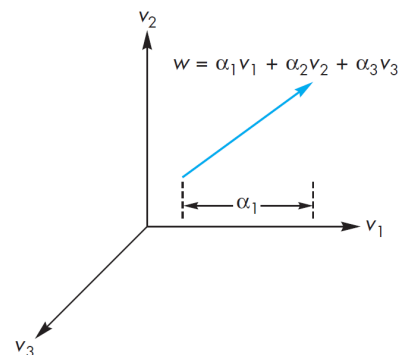
标架: 向量没有位置信息, 如何用向量坐标系表示点

标架 = 原点 + 基向量 (对应点 = 原点 + 位移)

例子: 在标架 (P_0, e_1, e_2, e_3) 下, 一个点 P 可以表示为 $P = P_0 + xe_1 + ye_2 + ze_3$, 对应坐标系 (x, y, z)

其实点和向量会混淆: 在普通 3 维坐标里, 点和向量都写成 (x, y, z)

——引入**齐次坐标**: 增加第 4 维 w 区分位置和位移, **点是 1, 向量是 0**



齐次坐标是所有计算机图形系统的关键

简答题：为什么要引入齐次坐标？

1. 增加第四维区分点和向量，点是 1，向量是 0

2. 点和向量的平移能通过矩阵乘法完成，4x4 矩阵下统一了旋转、缩放、平移

在三维空间中，我们希望所有的变换（旋转、缩放、平移）都能通过矩阵乘法（线性变换）来统一处理

问题：在标准的 3x3 矩阵下，线性变换只能表示旋转和缩放，**平移本质**

上是向量加法 ($x' = x + \text{Delta}$)，无法

用 3x3 矩阵乘法直接表示

$$\mathbf{v}' = S \cdot \mathbf{v} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & s_z \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} s_x \cdot x \\ s_y \cdot y \\ s_z \cdot z \end{bmatrix}$$

缩放：只涉及矩阵的对角线元素

旋转：以 **绕 Z 轴旋转** 为例

矩阵乘法凑不出平移的常数项，**四维相当于增加常数项维度，w=1 表示常数项系数=1**

$$\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

点：位置
向量：偏移，
平移无效

$$\begin{bmatrix} 1 & 0 & 0 & dx \\ 0 & 1 & 0 & dy \\ 0 & 0 & 1 & dz \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x + dx \\ y + dy \\ z + dz \\ 1 \end{bmatrix} \quad \begin{bmatrix} 1 & 0 & 0 & dx \\ 0 & 1 & 0 & dy \\ 0 & 0 & 1 & dz \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 0 \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \\ 0 \end{bmatrix}$$

坐标系的变换 3x3 矩阵：同一向量在不同基下坐标不同

考法：已知**新基** $b = [b_1 \ b_2 \ b_3]^T$ 在**旧基** $a = [a_1 \ a_2 \ a_3]^T$ 下的坐标（或反过来），问同一向量在新基下的坐标

解法：我想把 U 坐标系下的点 B 转换到 V 坐标系下，我只需要知道 B

坐标系的轴（基向量）在 A 里长什么样（已知 B 和 U）

(1) 写出基向量关系：已有坐标 U 变换到对方坐标

$$\begin{aligned}u_1 &= \gamma_{11}v_1 + \gamma_{12}v_2 + \gamma_{13}v_3 \\u_2 &= \gamma_{21}v_1 + \gamma_{22}v_2 + \gamma_{23}v_3 \\u_3 &= \gamma_{31}v_1 + \gamma_{32}v_2 + \gamma_{33}v_3\end{aligned}$$

$V(u=rv)$

(2) 构造变换矩阵 M

$$M = \begin{bmatrix} \gamma_{11} & \gamma_{12} & \gamma_{13} \\ \gamma_{21} & \gamma_{22} & \gamma_{23} \\ \gamma_{31} & \gamma_{32} & \gamma_{33} \end{bmatrix}$$

(3) $U^T = MV^T \rightarrow U = VM^T \rightarrow Va = Ub \rightarrow a = M^T b$

细节：我要从 V 变到 U，那我基向量是构造 $u=rv$

标架的变换：不仅换基，还可能换原点（于是需要 4×4 ）

$$u_1 = \gamma_{11}v_1 + \gamma_{12}v_2 + \gamma_{13}v_3$$

$$u_2 = \gamma_{21}v_1 + \gamma_{22}v_2 + \gamma_{23}v_3$$

$$u_3 = \gamma_{31}v_1 + \gamma_{32}v_2 + \gamma_{33}v_3$$

$$Q_0 = \gamma_{41}v_1 + \gamma_{42}v_2 + \gamma_{43}v_3 + P_0$$

$$M = \begin{bmatrix} \gamma_{11} & \gamma_{12} & \gamma_{13} & 0 \\ \gamma_{21} & \gamma_{22} & \gamma_{23} & 0 \\ \gamma_{31} & \gamma_{32} & \gamma_{33} & 0 \\ \gamma_{41} & \gamma_{42} & \gamma_{43} & 1 \end{bmatrix}$$

列向量左乘!!!

OpenGL 中的 6 个标架：

坐标用齐次坐标，标架之间用仿射变换矩阵实现

模型坐标系 Model $\rightarrow$ 世界坐标系 World $\rightarrow$ 相机坐标系 View(Camera)

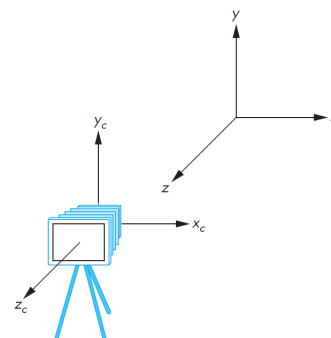
$\rightarrow$ 裁切坐标系 Clip $\rightarrow$ 规范化的设备坐标系 $\rightarrow$ 屏幕坐标系

OpenGL 图形管线中顶点通常在前三个坐标系间转换：

模型坐标系：物体自身的局部坐标（以物体为中心）

世界坐标系：把多个物体放在同一场景里建立空间关系

相机坐标系：从相机成像的坐标系（看世界）



OpenGL 中的相机模型：

默认状态：朝向-z 轴，头顶朝向+y 轴

世界坐标系与相机坐标系重合（相机在 origin，轴方向一致）

构建相机视图矩阵的数学步骤：

1. VRP: 相机在世界坐标系的 origin（相机的位置）
2. VPN: 利用 VRP - 目标点计算出 VPN（N 轴向量）
3. VUP: 视图竖直向量，表示相机的自身旋转姿态，利用 VUP 和 VPN 进行叉乘，计算出 U 轴向量

VPN+VUP 才能完全确定相机的方向

代码实现: `LookAt(eye, at, vup)`: 用代码定义一个相机

最终返回的 `mat4` 就是从“世界坐标”转换到“相机坐标”的矩阵，也就是

View Matrix, View 矩阵 = 相机在世界中的标架矩阵的逆（重点）

`mat4 LookAt(GLfloat eyex, GLfloat eyey, GLfloat eyez,`

`GLfloat atx, GLfloat aty, GLfloat atz,`

`GLfloat upx, GLfloat upy, GLfloat upz)`

`eye`: 相机在世界坐标系中的位置（视点）

`at`: 相机正在看世界坐标系里的哪一个点

（目标点），`at` 决定了的视线方向

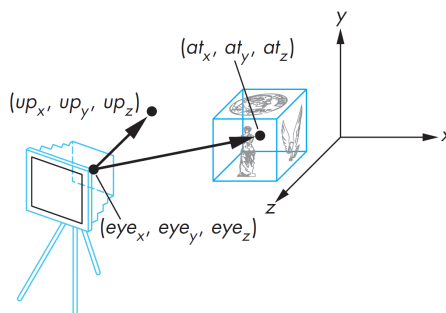
最终方向(向量)等于 `at-eye`

`Vup`: 向量（不是方向），相机的头顶朝向哪里（决定了相机的旋转角度）

——正着拍 (0, 1, 0) 意味着相机的头顶朝着世界坐标系的 Y 轴正方向

相机的移动与朝向: 相机坐标系由三个互相垂直的单位向量 **u**、**v**、**n** 构成

`eye` 是这个标架的原点



$$\mathbf{n} = \frac{\mathbf{vpn}}{|\mathbf{vpn}|}$$

n 轴（观察方向）：由 eye-at 得到（注意：OpenGL 中相机看向 -z，

所以 n 是看的反方向）

u 轴（右向量）：通过“上向量” v_{up} 与 n 轴进行叉乘得到

$$\mathbf{u} = \frac{\mathbf{v}_{\text{up}} \times \mathbf{n}}{|\mathbf{v}_{\text{up}} \times \mathbf{n}|}$$

v 轴（上向量）：通过 n 轴与 u 轴再次叉乘，确保三个轴完全正交

$$\mathbf{v} = \frac{\mathbf{n} \times \mathbf{u}}{|\mathbf{n} \times \mathbf{u}|}$$

易错：叉乘顺序很关键

回顾叉乘： $a = (a_x, a_y, a_z)$, $b = (b_x, b_y, b_z)$

$$a \times b = (a_y b_z - a_z b_y, a_z b_x - a_x b_z, a_x b_y - a_y b_x)$$

Section3 仿射变换

变换：把点（或向量）映射到另一个点（或向量）

仿射变换：定义为只有 12 个自由度的线性变换

$$C = \begin{bmatrix} \alpha_{11} & \alpha_{12} & \alpha_{13} & \alpha_{14} \\ \alpha_{21} & \alpha_{22} & \alpha_{23} & \alpha_{24} \\ \alpha_{31} & \alpha_{32} & \alpha_{33} & \alpha_{34} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

重点：保留线性结构，平行关系，线段内分比例

仿射 = 线性变换 + 平移

Section4 常见变换

平移变换：见上面分析齐次坐标

旋转变换：

2D 旋转公式（需背诵）： $x' = x \cos \theta - y \sin \theta$ $y' = x \sin \theta + y \cos \theta$

3D 旋转：绕 Z 轴：Z 坐标不变，XY 平面做 2D 旋转

核心口诀：X → Y → Z → X !!

$$\begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

$$R = R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

缩放变换：放缩变换必定有一个不动点

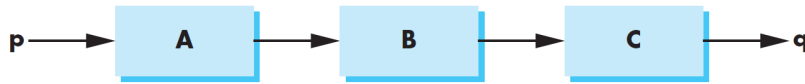
$$S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

错切变换：把面的不同区域向相反方向倾斜

刚体变换 = 平移 + 旋转，不改变大小

变换级联：从右向左计算

多个变换依次作用于点 p : $q = CBAP$



$$H_x(\theta) = \begin{bmatrix} 1 & \cot \theta & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

复杂旋转：

1. 绕任意“固定点”旋转：标准旋转矩阵 $R_z(\theta)$ 默认是绕着原点(0,0,0)转的

解决方案： $M = T(p_f) \cdot R(\theta) \cdot T(-p_f)$

(1) 平移：把物体的旋转中心移到原点 $\rightarrow T(-p_f)$

(2) 旋转：在原点放心大胆地转 $\rightarrow R_z(\theta)$

(3) 复位：把物体移回原来的位置 $\rightarrow T(p_f)$

2. 绕“任意轴”旋转：本质还是“先对齐、再旋转、再还原”

(1) 使用两个旋转变换 $R_x(\theta_x)$ 、 $R_y(\theta_y)$ 将轴向量 u 与 z 轴对齐

(2) 施加绕 z 轴的旋转变换 $R_z(\theta)$ (其他轴也行，一般都选 z 轴)

(3) 使用两个逆旋转变换 $R_y(-\theta_y)$ 、 $R_x(-\theta_x)$ 将向量 u 转至原位

$$R = R_x(-\theta_x) R_y(-\theta_y) R_z(\theta) R_y(\theta_y) R_x(\theta_x)$$

缩放与旋转在局部坐标系，平移在世界坐标系

总结矩阵求逆：

1. 纯旋转矩阵：直接求转置（角度制取负）

2. 纯平移矩阵：坐标取反

3. 纯缩放矩阵：缩放系数倒数

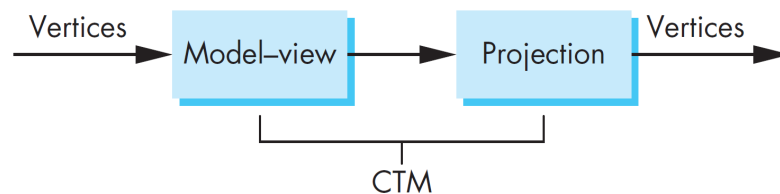
4. 旋转+平移（刚体变换）：分块公式——例子：View 矩阵

5. 缩放、错切、透视投影、TSR 组合：没辙

OpenGL 中的实现

CTM: 当前变换矩阵, $CTM = P \times MV$ (投影矩阵 $\times$ 模型视图矩阵)

是一个 4×4 齐次矩阵状态, 用于把顶点从当前坐标系变到后续处理需要的坐标系; 概念上它通常是若干变换 (模型/视图/投影等) 级联后的结果



OpenGL 的基本操作:

`RotateX(float xangle);` //绕 X 轴旋转

`RotateY(float yangle);` //绕 Y 轴旋转

`RotateZ(float zangle);` //绕 Z 轴旋转

`Translate(float dx, float dy, float dz);` //平移

`Scale(float sx, float sy, float sz);` // 缩放

考点: 使用这些基本操作来写伪代码

两个核心矩阵 :

1. `GL_MODELVIEW` (模型视图): 管“物体摆放” (Model) 和 “相机摆放” (View)

对应操作: `Translate`, `Rotate`, `Scale`

2. `GL_PROJECTION` (投影): 管“镜头选择” (`Perspective/Ortho`)

决定了是近大远小还是平行投影

例子: 绕固定点旋转[平移到原点 -> 旋转 -> 还原]

操作: `ctm = Translate(4.0, 5.0, 6.0)*R* Translate(-4.0, -5.0, -6.0);`

代码实现的两种流派

(1) CPU 计算：慢

(2) 顶点着色器计算： $gl_Position = ModelViewProjection * vPosition$

把变换矩阵作为一个 uniform 变量传给 Shader —— 先乘 ModelView 变成眼坐标，再乘 Projection 变成裁剪坐标

记住!!! $gl_Position$ 的计算 = $Projection * View * Model$

优点：GPU 并行计算，极快

简答：向量用“坐标系/基”就能表示；点为什么必须引入“标架（原点 + 基）”才能表示？

向量表示的是位移，不依赖原点，所以只需要一组基表示，而点表示位置，必须相对于某个原点，所以需要标架

简答：怎么理解 View 矩阵 = 相机模型矩阵的逆？

在 OpenGL 的渲染管线里，相机永远固定在原点 (0,0,0)，且永远看向 -Z 方向，为了模拟“相机移动到了 (10, 10, 10) 并且向右旋转了 30 度”，我们需要把整个世界的所有物体，先向左旋转 30 度，再移动到 (-10, -10, -10)

所有的操作都是反着来的，这就是“逆矩阵”的物理含义

相机标架矩阵 M_{cam} 作用：它把一个点从相机坐标系变换到世界坐标系

$$P_{world} = M_{cam} \cdot P_{camera}$$

渲染时实际需要的矩阵是他的逆矩阵，因为我们只知道物体在世界里的位置（比如房子盖在坐标 (100, 0, 0)），我们需要算出它相对于相机在哪里（为了画在屏幕上） $View = M_{cam}^{-1}$ ，而 LookAt 函数返回的就是 View 矩阵

$$P_{camera} = M_{cam}^{-1} \cdot P_{world}$$

更进一步，View 矩阵的原理：先平移，后旋转

第一步：平移，相机的观察点平移到世界原点 (0,0,0)，对应逆操作是世

界坐标系 Translate(-eye)

第二步：旋转

$$T = \begin{bmatrix} 1 & 0 & 0 & -e_x \\ 0 & 1 & 0 & -e_y \\ 0 & 0 & 1 & -e_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

xyz -> uvn 坐标系变换，这个矩阵叫视图定向矩阵

最终结果：View = R × T

$$View = \begin{bmatrix} u_x & u_y & u_z & -\mathbf{eye} \cdot \mathbf{u} \\ v_x & v_y & v_z & -\mathbf{eye} \cdot \mathbf{v} \\ n_x & n_y & n_z & -\mathbf{eye} \cdot \mathbf{n} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R = \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ n_x & n_y & n_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

第四章 观察

投影目的：设置镜头——设置投影矩阵，规范化视见体

OpenGL 默认的投影矩阵是一个 单位矩阵

简答：为什么要投影？

一方面是将三维场景中的对象映射到二维投影平面上，形成最终的图像，

通过平行投影或透视投影两种方式模拟不同视觉效果

另一方面投影变换将相机坐标系变换到裁切坐标系，定义视见体，投影归一化简化裁切运算等后续操作

视图的三个基本要素（必背）：

- (1) 对象：被观察的物体
- (2) 观察者：通常简化为投影中心 (COP) + 投影面
- (3) 从对象到投影平面的投影变换

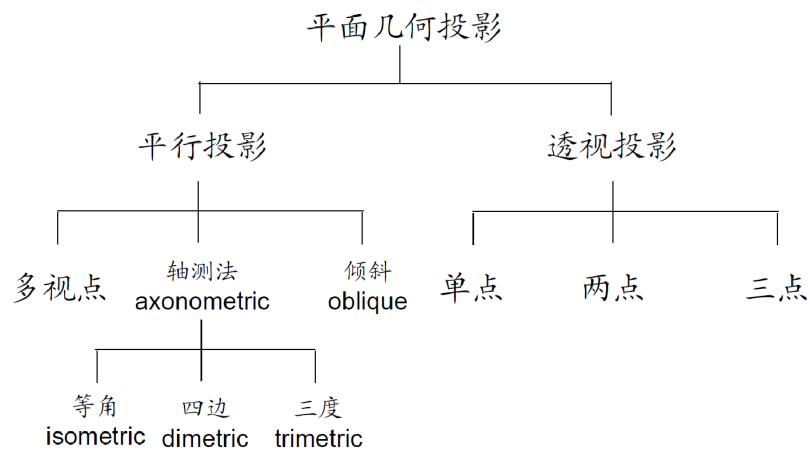
概念：基准面和投影平面

基准面：依附于物体本身的，用来衡量物体摆放姿态

投影平面：客观存在的（比如屏幕、纸张），是用来接收图像的

两大投影体系：透视 vs 平行，最核心的分类区别在于“投影中心”的位置

记忆类型：透视投影 一点二点三点



平行投影：

定义：投影中心在无穷远处，投影线彼此平行

正交投影：投影线 垂直 于投影平面

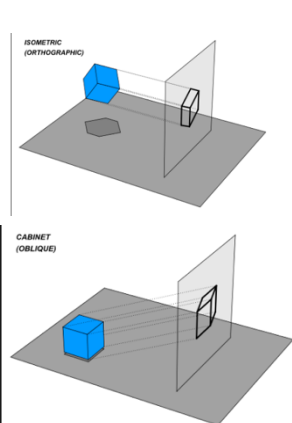
- 多视点投影：如三视图（主、俯、左），保留真实形状和角度
- 轴测投影：投影面与物体三个主面都有夹角（能同时看到三个面），如等轴测图

斜投影：投影线 不垂直 于投影平面

- 优势：在平行于投影面的面上，角度和形状完全保留，同时又能显示出侧面

透视投影：

定义：投影中心在有限距离，核心特征是近大远小



灭点：现实中平行的线在透视图中的会汇聚于一点

- **三点透视：**没有基准面平行于投影面，有 3 个灭点
- **两点透视：**有一个基准方向（如垂直棱）平行于投影面，有 2 个灭点
- **一点透视：**有一个基准面（如建筑正面）平行于投影面，有 1 个灭点

优缺点：视觉效果真实（适合动画、建筑效果图），但不能用于测量尺寸



判断：（1）竖直是否聚合 （2）左右阶梯是否聚合 （3）前后阶梯是否聚合

投影规范化：把所有的投影都转化为正交投影

问题：视见体形状各异（正交投影是长方体，透视投影是棱台），如果针对每种形状都设计一套裁切算法太麻烦

- **视见体：**3 维区域，裁切需要一个有限体积

解决方案：先变形，再裁切，通过一个矩阵变换，把任意形状的视见体统统“挤压”或“拉伸”成一个标准的 $2 \times 2 \times 2$ 立方体（即规范视见体，范围 $x, y, z \in [-1, 1]$ ），等价于把对象变形使得它的正交投影图与原来想要得到的对象的投影图相同

好处：硬件只需要对这个标准立方体进行裁切，**算法统一且高效**

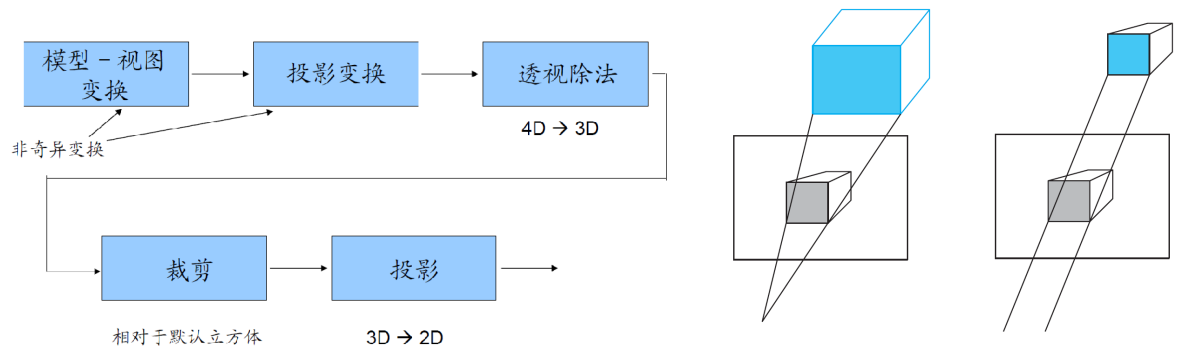
类比模视转换：投影变换（规范化/变形）

逻辑：裁切盒子不动（永远是标准的 $2 \times 2 \times 2$ 立方体）——**相机不变**

操作：通过让对象变形（投影规范化），让原本在奇怪视见体（比如棱台）里的物体挤进这个标准立方体里 —— **等同于让世界旋转**

结果：从相机坐标系 变换到 裁切坐标系

整个过程：



投影变换 $\neq$ 投影

投影变换：还在 4 维齐次坐标空间，核心作用是把物体变形，深度信息 z

依然被完整保留

透视除法：关键一步，处以深度信息 w 从 4D 到 3D

裁切：对标准立方体进行正交投影裁切算法

投影：这里才是最后的 3D 降维为 2D

简答：为什么不直接模视转换就投影？

尽可能的保留深度信息，用于后续的隐藏面消除

投影变换：正交投影 or 透视投影

函数本质是在“用参数定义视见体”，返回投影矩阵，

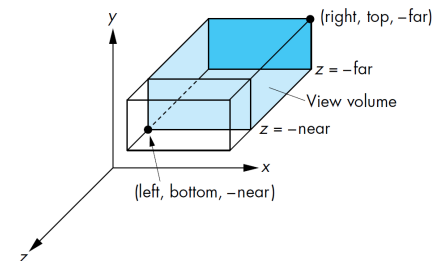
用于投影规范化，类似 LookAt 函数返回 View 矩阵

正交投影的实现 (Ortho)：投影平面 $z=0$

函数：mat4 **Ortho**(left, right, bottom, top, near, far)

注意符号：左下角(left,bottom,-near) 右下角(right,top,-far) 相机指向-z

矩阵推导： $M=ST$ （先平移后缩放）



1. 平移：把长方体的中心移到原点

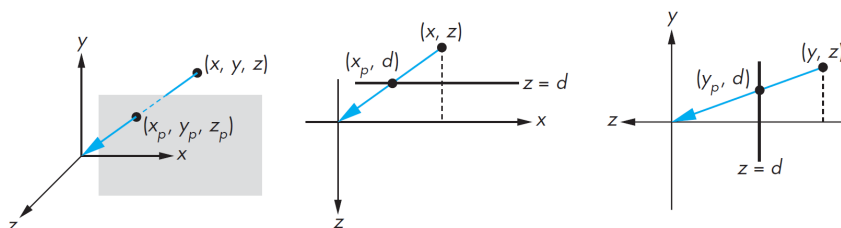
$$T\left(-\frac{left + right}{2}, -\frac{bottom + top}{2}, \frac{near + far}{2}\right)$$

2. 缩放：把长宽高缩放到 $[-1, 1]$ 的范围

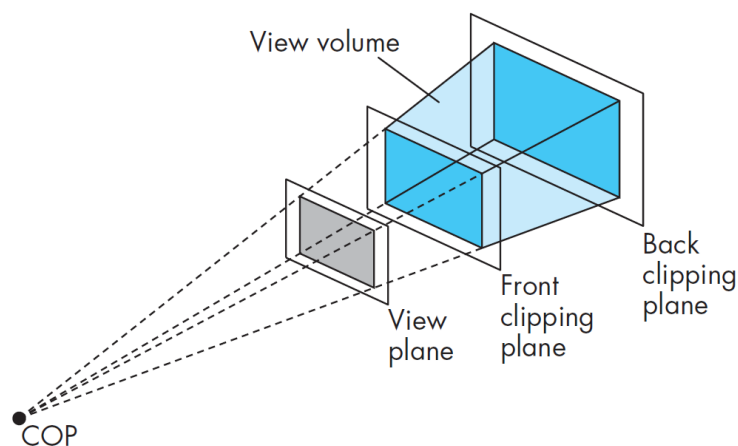
$$S\left(\frac{2}{right - left}, \frac{2}{top - bottom}, -\frac{2}{far - near}\right)$$

透视投影的实现：投影平面 $z=d$

透视方程：等比缩放 $x_p = \frac{x}{z/d}, y_p = \frac{y}{z/d}, z_p = d$



视见体：半无穷的棱台



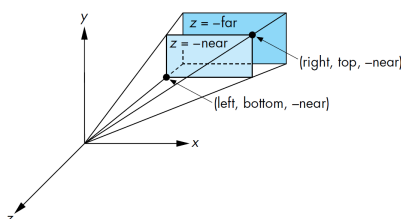
两个关键函数：Frustum & Perspective

- **Frustum**：视见体函数，最原始的定义方式。通过指定 $left, right, bottom, top, near, far$ 六个面来定义棱台

它允许定义不对称的视见体（比如 $left$ 不等于 $-right$ ）

没有 Far 的四个坐标：不需要去定义远平面有多大，它可以通过 $near$ 平面和原点的比例推导得到

- **Perspective**：透视函数，更符合相机直觉 ($fovy, aspect, near, far$)



fovy (Field of View Y): 视野角度。想象你眼睛张开的角度, 角度越大, 看到的范围越广 (广角镜头)

aspect: 长宽比。视见体横截面的宽度除以高度, 通常和屏幕长宽比一致

参数换算关系: $top = near \times \tan(fovy)$, $right = top \times aspect$

它生成的视见体是对称的, 所以只需要一边的参数

透视归一化: 棱台 $\rightarrow$ 标准立方体

核心公式: $z'' = -(\alpha + \beta/z)$

变换矩阵 N: $w = -z$ 【透视除法要求】

$$\alpha = -\frac{far + near}{far - near} \quad \beta = -\frac{2 * far * near}{far - near}$$

$$N = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \alpha & \beta \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

函数对应的矩阵:

- Frustum: 先变成 对称+标准 的棱台, 再透视归一化

1. 错切 H: 不对成棱台变成对称
2. 缩放 S: Fovy 和 aspect 不一致 $\rightarrow$ 棱台的侧面变成 $x=\pm z$ 和 $y=\pm z$
3. 透视归一化 N

$$P = NSH = \begin{bmatrix} \frac{2*near}{right-left} & 0 & \frac{right+left}{right-left} & 0 \\ 0 & \frac{2*near}{top-bottom} & \frac{top+bottom}{top-bottom} & 0 \\ 0 & 0 & -\frac{far+near}{far-near} & \frac{-2far*near}{far-near} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

- Perspective 矩阵算出了这四个数, 底层还是去调用 glFrustum

在透视投影矩阵之后, w 会与深度相关 (通常和 $-z$ 成比例), 所以远处点的 w 更大, x/w 、 y/w 就更小, 于是产生近大远小

阴影的投影矩阵

投影矩阵核心口诀：第四行决定“缩放源”

(1) 正交投影：投影平面 $z=0$ ，第三行全零，平行光不需要做除法，所以保持 0001

(2) 简单透视：投影平面 $z=d$ ，物体离得越远 (z 越大)，看起来越小，所以缩放源是 z ，最后靠除法来变小，所以保持 xyz 不变， $w=z/d$ ，所以第四行是 $0\ 0\ 1/d\ 0$

(3) 阴影投影：光源在头顶 (Y 轴上，高度为 y_l)，“竖着放”的透视投影，缩放源是高度 y ， y 越接近 y_l ，投影越远，对应 $-y_l$

特殊之处：投影中心在 (x_l, y_l, z_l) ，需要先平移到原点再投影

简单的投影矩阵公式，默认投影中心（光源）必须在原点 $(0,0,0)$

现实情况是：光源在任意位置 (x_l, y_l, z_l) ，而地面在 $y=0$

$$M_{shadow} = T(x_l, y_l, z_l) \cdot M_{simple} \cdot T(-x_l, -y_l, -z_l)$$

1. 先投影到原点
2. 透视矩阵+透视除法
3. 平移回光源位置

$$M_{final} = \begin{bmatrix} -y_l & x_l & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & z_l & -y_l & 0 \\ 0 & 1 & 0 & -y_l \end{bmatrix}$$

区分：两种投影矩阵

1. 相机投影矩阵 (Projection 层)

Ortho / Frustum / Perspective 等 API 返回的 4×4 矩阵属于相机

投影矩阵 P 。其作用是把相机坐标系中的视见体（正交的盒子/透视的棱台）映射到统一的裁剪空间，实现投影规范化，便于后续的裁剪、透视除法以及深度测试等标准管线操作。标准顶点变换链为：

$$p_{clip} = P V M p_{obj}$$

2. 几何投影矩阵（几何变换/Model 层）

某些“简单投影矩阵”（例如把点正交投到某平面、平面阴影把物体沿光线方向压到地面平面）本质上是对几何本身的变换，可视为一种特殊的模型变换矩阵 S ，与平移/旋转/缩放同类，通常组合到模型矩阵中：

$$M' = S M$$

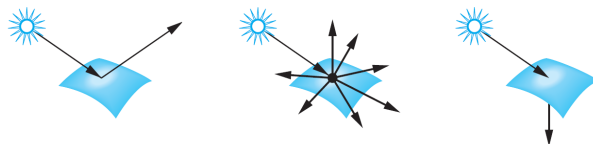
Chapter5 明暗绘制

光线与材质

某一点的颜色取决于光照与材质之间的多次作用

相互作用的三大分类：

1. 镜面反射：一部分入射光线被吸收，但是所有反射出去的光线都沿着同一个方向【入射角=反射角】
2. 漫反射：向各个方向散射的光线强度都相等
3. 折射：半透明表面，允许入射光线的一部分进入表面并且从对象的另一个位置再发射出去

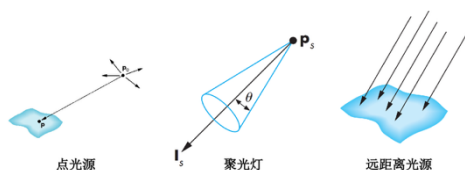


光亮度：

在大多数应用中，可以用 RGB 三种成分的强度表示光源亮度

光亮度函数为 $I = [I_r, I_g, I_b]$

简单光源模型



基本光源:

$$I_a = \begin{bmatrix} I_{ar} \\ I_{ag} \\ I_{ab} \end{bmatrix}$$

- a) 环境光: 环境中每个位置的环境光强度完全相同, 场景中

光源和对象之间多次反射导致

- b) 点光源: 向各个方向发射的光线强度相等, 点 p 接收的光亮度反
比于光源与点的距离的平方

$$i(p, p_0) = \frac{1}{|p - p_0|^2} I(p_0)$$

$$I(p_0) = \begin{bmatrix} I_r(p_0) \\ I_g(p_0) \\ I_b(p_0) \end{bmatrix}$$

- c) 聚光灯: 具有一个比较窄的照明范围, 通常为圆锥形半无穷区域, 如果 $\theta = 180^\circ$, 聚光灯成为点光源, 逐渐衰减

$$p_0 = \begin{bmatrix} x \\ y \\ z \\ 0 \end{bmatrix}$$

- d) 远距离光源: 在无穷远处的光源, 光线为平行线, 向量为常数

反射模型: 抽象层面, 反射光强度 $I =$ 入射光源强度 $L \times$ 反射率 R

反射率取决于曲面材质、光线方向(入射出射)、光源距离、视点位置

Phong 反射模型(核心算法): 能手写出计算公式, 并理解每一项的物理含义

计算需要四个向量: 曲面法向量 n , 视线方向 v , 入射方向 l , 反射方向 r

Phong 里 物体表面反射光由三部分组成: 环境光 + 漫反射 + 镜面反射

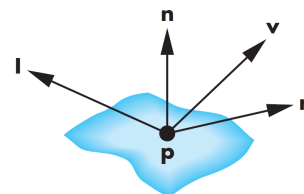
- (1) 环境光: $I_a = k_a L_a$, $0 \leq k_a \leq 1$, 反射率由环境光反射系数 k_a 确

定, 与观察角度、光源位置无关, 物体表面所有点的环境光强度相同

- (2) 漫反射:

- a) Lambert 定律: 光强取决于入射光与法线的夹角。光线越垂直于
表面 (夹角越小), 表面越亮

- b) 漫反射光强公式: $I_d = k_d L_d \max(l \cdot n, 0)$, 与视线方向 v
无关, 和曲面法向量有关, 无论从哪看亮度都一样



- (3) 镜面反射

a) 公式: $I_s = k_s L_s \max(r \cdot v, 0)^\alpha$, $r \cdot v$: 反射光 r 与视线 v 的夹角余弦, 与视线方向密切相关, 只有当视线看向反射方向附近时才能看到高光

α : 高光系数, 越大, 高光越集中、越锐利

(4) 距离衰减: $\frac{1}{a+bd+cd^2}$ d 是距离, 常数与线性项使照明效果更加柔和

最终公式: 和光源距离有关——衰减因子

$$I_s = \frac{1}{a + bd + cd^2} (k_d L_d \max(l \cdot n, 0) + k_s L_s (\max(r \cdot v, 0))^\alpha) + k_a L_a$$

改进的 Phong 模型:

Phong 模型中, 镜面反射要计算反射向量 r 计算量较大

Blinn-Phong 模型: 记住改进和好处

- **改进:** 不计算反射向量, 而计算半程向量 $h = \frac{l+v}{|l+v|}$ 即光线方向 l 和视线方向 v 的中间方向, 转换为法向量 n 和半程向量 h 的点乘 $(n \cdot h)^\beta \rightarrow (r \cdot v)^\alpha$
- **好处:** 计算效率更高—计算 h 只需要向量加法和一次归一化, 速度更快
光照效果更稳定, 过渡更自然

更符合物理规律

球体某一点法向量:

求法向量方法: 对每个点求梯度

$$\mathbf{n} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \\ \frac{\partial f}{\partial z} \end{bmatrix} = \begin{bmatrix} 2x \\ 2y \\ 2z \end{bmatrix} = 2\mathbf{p}$$

OpenGL 明暗处理的标准流程

1. 启用明暗处理功能并选择模式
2. 计算/指定法向量
3. 指定光源
4. 指定材料属性

光源参数：环境光分量、漫反射分量、镜面反射分量、光源位置、衰减因子

材质参数：环境光，漫反射，镜面反射三个系数、高光因子、自发光

光照模型程序实现、顶点着色器实现

```
//计算四个归一化向量
vec3 N = normalize(norm);
vec3 L = normalize(l_pos - pos);
vec3 V = normalize(-pos);
vec3 R = reflect(-L, N);

//计算环境光光照
color4 ambient = AmbientProduct;

//计算漫反射光光照
color4 diffuse = DiffuseProduct * max(dot(N, L), 0.0);

//计算镜面反射光光照
color4 specular = SpecularProduct *
pow(max(dot(V,R),0.0), material_shininess);

//光照求和
color = ambient + diffuse + specular;
Color.a = 1.0;
```

```
void main()
{
    // 计算顶点最终位置，传给顶点着色器
    gl_Position = ModelViewProjection * vPosition;

    // 将顶点坐标、光源坐标和法向量转换到相机坐标系
    vec3 pos = (ModelView*vPosition).xyz;
    vec3 l_pos = (ModelView*LightPosition).xyz;
    vec3 norm = (ModelView*vec4(vNormal, 0.0)).xyz;

    //计算四个归一化向量
    ...

    //计算镜面反射光光照
    // 注意如果光源在背面则去除高光
    if( dot(L, N) < 0.0 ) {
        specular = vec4(0.0, 0.0, 0.0, 1.0);
    }
    ...
}
```

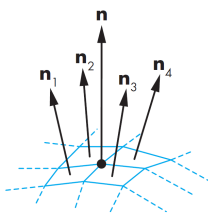
多边形着色方法 (Shading)

- 均匀着色：只算一次颜色，整个面颜色相同，缺点是明显的块状感，Mach 带效应
- Gouraud 着色/光滑着色：在顶点处算颜色，中间像素通过颜色插值得到，计算量小但是可能会丢失高光

计算步骤：

(1) 求顶点法向：对于共享同一个顶点的多个多边形（面），取它们法向量的平均值，作为该顶点的法向

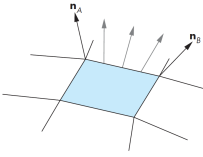
$$\mathbf{n} = \frac{\mathbf{n}_1 + \mathbf{n}_2 + \mathbf{n}_3 + \mathbf{n}_4}{|\mathbf{n}_1 + \mathbf{n}_2 + \mathbf{n}_3 + \mathbf{n}_4|}$$



(2) 算顶点颜色：利用 Phong 反射模型公式 ($I = I_a + I_d + I_s$)，在每个顶点处计算光照颜色

(3) 插值颜色：对于多边形内部的像素点，直接利用计算好的顶点颜色进行双线性插值，从而填满整个多边形

- Phong 着色：根据每个顶点的法向，插值出多边形内部各点的法向，然后基于光照模型计算出各点的颜色 $\mathbf{n}(\alpha, \beta) = (1 - \beta)\mathbf{n}_C + \beta\mathbf{n}_D$



有效地降低 Mach 带效应，图形更光滑，但是计算量大

1. 求顶点法向：和 Gouraud 方法一样，先求出顶点的平均法向
2. 插值法向：关键区别，对于多边形内部的每一个像素，不是插值颜色，而是根据顶点的法向量，线性插值出该像素点对应的法向量，注意要重新归一化

3. 算像素颜色：有了当前像素的法向量后，对每一个像素（片元）单独应用 Phong 反射模型公式计算最终颜色

Gouraud 着色：在顶点着色器计算颜色

Phong 着色：在片元着色器计算颜色

真题 1: Gouraud 着色与 Phong 着色的对比

相同点：两种着色第一步都是计算顶点的平均法向，数据依赖相同，都需要特定的数据结构来表示网格，以便找出与顶点相邻的多边形来求平均法向

不同点：Gouraud 着色是在顶点上计算光照颜色，然后对颜色进行线性插值求出多边形内部各点的颜色；Phong 着色是对顶点的法向量进行线性插值，求出多边形内部每个片元的法向量，然后对每个片元独立应用光照模型计算颜色，所以 Phong 着色得到的曲面看起来更光滑，高光效果更真实，能有效解决 Mach 带效应，但是计算量更大，速度慢

流水线策略

图形绘制主要关注两个元素：**几何图元**和**像素**

两种实现循环：

1. **基于对象空间 (Object Space)**: for(each_object), 对于每个对象, 确定它所覆盖的像素, 并用对象的状态确定像素亮度, 缺点是**难以处理全局效果 (如阴影、反射)**
2. **基于图像空间 (Image Space)**: for(each_pixel), 对于每个像素, 确定投影到这个像素的离观察者最近的那个对象, 从而基于该对象计算像素的亮度, 优点是适合处理全局效果, 但**无法事先确定哪些图元影响哪些像素, 计算量大**

图形绘制系统的 4 个主要任务: 建模 → 几何处理 → 光栅化 → 片元处理

几何处理:

目的: 确定哪些几何对象要被显示, 并把明暗值或者颜色值赋给其顶点, 处理三维顶点 (浮点运算)

四个核心步骤: 投影 (模视转换 → 投影规范化) → 图元组装 (顶点组合成对象) → 裁剪 → 明暗处理 (光照)

光栅化:

定义: 基于顶点 给出 表示对象的一组像素

具体: 将归一化坐标转化为屏幕坐标 (视口变换), 并生成片元

片元处理: 处理颜色、纹理映射、隐藏面消除、混合 (透明)、反走样

裁剪——掌握核心算法

目的: 确定哪些图元在视见体内, 去除不可见部分

Cohen-Sutherland 算法 (线段裁剪) —— CS 算法

核心思想：编码，将空间划分为 9 个区域，用 4 位二进制码表示端点位置

置

编码规则：上(1000)、下(0100)、右(0010)、左(0001)

判断：

1001	1000	1010	$y = y_{\max}$
0001	0000	0010	
0101	0100	0110	$y = y_{\min}$
$x = x_{\min} \quad x = x_{\max}$			

1. $\text{Code1} \mid \text{Code2} == 0$ (两个都是 0000)：完全在内部，接受
2. $\text{Code1} \& \text{Code2} != 0$ (某一位同时等于 1)：都在某一侧外部，抛弃
3. 其他情况：求交点，缩短线段后递归处理

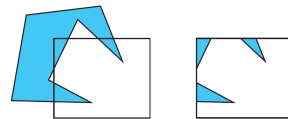
Liang-Barsky 算法 (线段裁剪) —— LB 算法

核心思想：参数化表示线段，计算入点和出点的参数 α

$$p(\alpha) = (1 - \alpha)p_1 + \alpha p_2, \quad 0 \leq \alpha \leq 1$$

优势：相比 Cohen-Sutherland 减少了重复的求交计算，效率更高

Sutherland-Hodgeman 算法 (多边形裁剪)：



剪凸多边形最多得到一个凸多边形（两个凸集的交集仍然是凸集）；非凸

多边形常先剖分（三角化）

流水线结构：将多边形依次通过四个边界的裁剪器（左、右、上、下）

输出：裁剪后的一组新顶点，对窗口一边进行裁剪时，与其它边无关

光栅化——掌握核心算法

直线光栅化：线段怎么变成像素点序列

- DDA 算法：基于微分方程，利用浮点数累加斜率 m ；缺点是涉及浮

点运算和取整，效率较低

只对 $0 \leq |m| \leq 1$ 的直线应用上述算法， $|m| > 1$ 互换 x 和 y

- **Bresenham 算法**：考试高频点，死记

优势：只使用整数加减法，没有浮点运算，更快更稳

原理：利用决策变量 d 判断下一个像素是选上方还是下方（基于中点位置判断）

多边形填充：OpenGL 只保证正确填充凸多边形

内外检测：真题：相交检测判断内外

1. 奇偶测试法：射线法，交点为奇数在内，偶数在外，交点落在顶点要特别处理

2. 环绕测试法（计算环绕数）：射线取向右的水平射线，计算穿过次数，逆时针算 $+1$ ，顺时针算 -1

- **漫水填充**：先用 Bresenham 算法光栅化边界，再从种子点开始递归填充颜色

- **扫描线填充**：利用边表和活性边表，按扫描线顺序填充，效率较高

隐藏面消除

基于对象空间：

1. 画家算法：像画家一样，先画远的，后画近的（覆盖），需要对多边形深度排序，效率为 $O(n \log n)$

缺点是无法处理循环重叠（A 挡 B，B 挡 C，C 挡 A）

2. 背面剔除：通过法向量 n 和视线向量 v 的点积判断。如果 $v \cdot n > 0$ （即夹角小于 90° ），可见（保留），视线向量指向人眼

基于图像空间：

Z-Buffer (深度缓冲) 算法：考试核心。

原理：为每个像素维护一个两块内存空间

1. **颜色缓冲区：**用于存储当前像素点显示的**颜色或亮度值**
2. **深度缓冲区：**用于存储当前像素点对应的最近物体的**深度值 z**

替换规则：绘制新像素时，比较每个多边形深度与深度缓冲中的值，**如果新像素深度更小，则更新颜色和深度**

优势：硬件易实现，不需要对物体排序，复杂度 $O(nmk)$ ($n \times m$ 分辨率， K 个多边形比较)，Z-Buffer 完美解决了**循环重叠和贯穿问题**，因为它是在像素级进行深度判定，而不是物体级

性能优化：由于多边形是平的，当你沿着一行像素（即扫描线）从左向右移动时，深度 z 的变化是线性的，能用简单的加法代替复杂的乘除法来确定每个点的深度值

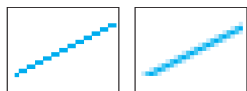
Z 缓冲区扫描线算法：

掌握思想，这一行只穿过一个多边形时不需要进行任何深度比较，直接画颜色就行，同时穿过了**多个多边形**（发生重叠）时，程序才会调取深度信息来决定谁在前面

反走样：减少锯齿感，使图形边缘看起来更平滑

走样：理想光栅化直线应是 1 像素宽，但由于像素离散化，会导致斜线出现锯齿

方法：面积平均法，对每个 x ，把**理想直线（假设）覆盖的像素面积**乘以**颜色**，实现灰度/颜色过渡

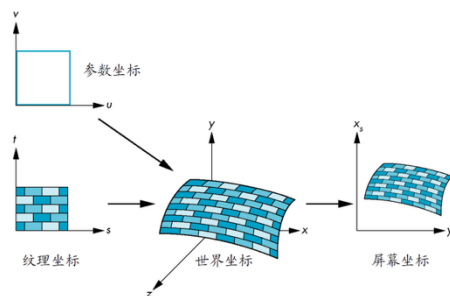


纹理映射：将 2D 图像映射到 3D 物体表面

为了解决几何建模过于复杂而无法实时绘制，通过纹理贴图来增加细节

坐标系转换：

1. 纹理坐标系 (s,t) ：表示纹理
2. 参数坐标系 (u,v) ：绘制曲面
3. 世界坐标系 (x,y,z) ：真正发生映射
4. 屏幕坐标系 (x_s,y_s) ：图像显示



映射方法：

1. **逆映射**：给定屏幕像素 (x,y) -> 找物体上的点 (x,y,z) -> 找纹理坐标

(s,t) ，屏幕到世界通过光栅化+深度可以轻松实现，**难在世界到纹理**

函数实现：

世界和参数关系：对于参数曲面来说，世界坐标是由参数 (u, v) 生成的

- a) **线性映射**：利用曲面的参数方程来建立联系

$u = as + bt + c, v = ds + et + f$ ，实现简单但没考虑曲面弯曲

- b) **两步映射**：首先把纹理映射到一个简单的中间曲面，再从中间对象到实际对象的映射

OpenGL 实现：

创建纹理对象 (`glGenTextures`) -> 绑定 (`glBindTexture`) -> 指定图像数

据 (`glTexImage2D`) -> 设置参数 (`glTexParameter`) -> 传递坐标（顶点着色器，渲染阶段）—— 最终颜色（片元着色器）：融合 or 贴图

层级建模!!!

基础概念——图符与实例

图符 -- Class: 也叫图元, 模型最原始的定义, 模型坐标系里定义, 最小完备定义——在内存中只有一份

实例 -- Object: 将图符放入场景 (世界坐标系) 后形成的对象

- 同一个图符 (如一个轮子) 可以在场景中出现多次, 每次出现就是一个不同的“实例”

实例变换: 几何变换矩阵 把图符变成实例 (模型坐标系 -> 世界坐标系)

先缩放调整大小, 再旋转调整方向, 最后平移调整位置 $M = TRS$

必须严格遵守 TRS 顺序 保证物体不变形 —— 反例: 非均匀缩放

图符-实例表: 简单的存储方式

列出所有图符, 然后列出每个实例对应的变换参数 (S, R, T), 无法表达物体部件之间的关联关系 (例如: 车动了, 轮子必须跟着动) —— 层级建模

例子: 车身和轮子

为什么要层级建模——当你移动车身时, 必须手动计算每个轮子的新位置, 非常繁琐且容易出错

解决方案: 建立父子关系, 车身是父节点, 轮子是子节点

子节点的运动 = 父节点的运动 + 子节点自身的相对运动

数据结构: 树与图

树: 最常用的层级结构。根节点 (如车身) 连接子节点 (如轮子)

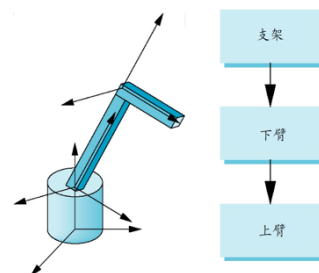
有向无环图 (DAG): 如果多个部件共用同一个图符 (例如四个轮子都共用同一个圆柱体模型), 结构就变成了 DAG。

但在逻辑处理上, 我们通常还是把它看作树来处理

重难点——机器人手臂

1. 模型的分解 一个简单的机器人手臂分为三部分，形成一个运动链：

- (1) 支架：只能在底座上旋转，**独立旋转**
- (2) 下臂：连接在支架上，随支架旋转，且**绕关节处**可以转动
- (3) 上臂：连接在下臂上，随前两者运动，自己也有关节



2. 变换矩阵的累积 (核心考点)

(1) 支架:

- 变换：只绕 Y 轴旋转 θ
- 矩阵： $M_{base} = R_y(\theta)$

(2) 下臂:

- 变换：先应用支架的旋转，再平移到关节处 l_a ，再进行旋转
- 矩阵： $M_{lower} = R_{base} \cdot T_{l_a} \cdot R_{l_a}$ **注意矩阵顺序**

(3) 上臂:

- 变换：继承下臂的所有变换，然后平移到第二个关节处，再旋转
- 矩阵： $M_{upper} = R_{base} \cdot T_{l_a} \cdot R_{l_a} \cdot T_{l_{ua}} \cdot R_{ua}$

OpenGL 实现

增量的方式改变模-视矩阵以体现运动联动性，**多级层级建模实现 —— 栈**

```
model_view = RotateY(Theta[Base]); //支架变换矩阵
base(); //绘制支架

model_view = model_view * Translate(0.0, BASE_HEIGHT, 0.0)
               * RotateZ(Theta[LowerArm]); //下臂变换矩阵
lower_arm(); //绘制下臂

model_view = model_view * Translate(0.0, LOWER_ARM_HEIGHT, 0.0)
               * RotateZ(Theta[UpperArm]); //上臂变换矩阵
upper_arm(); //绘制上臂
```

```
model_view = RotateY( theta[Torso] ); //躯干变换矩阵
torso(); //躯干绘制

mvstack.push( model_view ); //保存躯干变换矩阵
model_view *= 头结点局部变换矩阵;
head(); //头部绘制
model_view = mvstack.pop(); //恢复躯干变换矩阵

mvstack.push( model_view ); //保存躯干变换矩阵
model_view *= 左上臂局部变换矩阵;
left_upper_arm(); //左上臂绘制
model_view *= 左下臂局部变换矩阵;
left_lower_arm(); //左下臂绘制
model_view = mvstack.pop(); //恢复躯干变换矩阵

mvstack.push( model_view ); //保存躯干变换矩阵
model_view *= 右上臂局部变换矩阵;
right_upper_arm(); //右上臂绘制
model_view *= 右下臂局部变换矩阵;
right_lower_arm(); //右下臂绘制
model_view = mvstack.pop(); //恢复躯干变换矩阵
:
```